

AnchorIQ 시스템 동작 흐름 완벽 가이드

비전공자를 위한 CS 기초부터 실제 코드까지 — "사용자가 질문하면 답이 나오기까지 모든 것"

목차

- [Part 1: 인터넷 기초 — 브라우저에서 서버까지](#)
- [Part 2: HTTP 완전 정복](#)
- [Part 3: REST API란 무엇인가](#)
- [Part 4: Spring Boot & DDD 아키텍처](#)
- [Part 5: 보안 — JWT 인증 흐름](#)
- [Part 6: AnchorIQ AI 질의 전체 흐름 \(코드 레벨\)](#)
- [Part 7: 데이터베이스 & 온톨로지](#)
- [Part 8: Kafka 이벤트 파이프라인](#)
- [Part 9: 프론트엔드에서 백엔드까지 전체 연결](#)
- [Part 10: 운영 인프라 — Docker, Nginx, 모니터링](#)

Part 1: 인터넷 기초 — 브라우저에서 서버까지

1.1 사용자가 주소를 입력하면 무슨 일이 벌어지나?

사용자가 브라우저에 `http://localhost:3004` 를 입력하는 순간부터 화면이 뜨기까지의 과정입니다.

전체 흐름

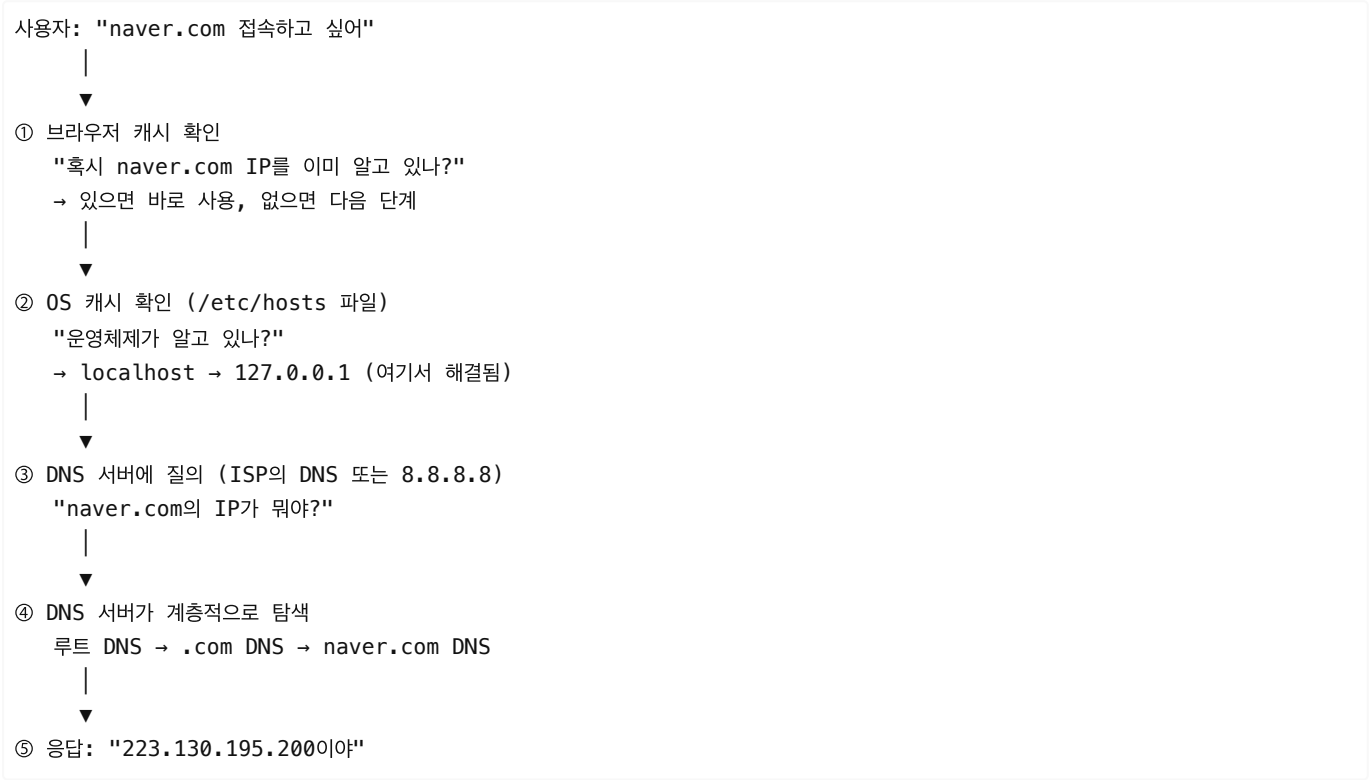
```
[사용자 브라우저]
|
| ① "http://localhost:3004" 입력
▼
[DNS 조회]
| "localhost"를 IP 주소로 변환
| localhost → 127.0.0.1 (내 컴퓨터)
▼
[TCP 연결]
| 3-Way Handshake (SYN → SYN-ACK → ACK)
| "통신할 준비 됐어?" → "됐어!" → "OK 시작!"
▼
[HTTP 요청 전송]
| GET / HTTP/1.1
| Host: localhost:3004
▼
[Next.js 서버 (포트 3004)]
| React 페이지를 HTML로 렌더링
▼
[HTTP 응답 반환]
| 200 OK
| Content-Type: text/html
| <html>... 대시보드 화면 ...</html>
▼
[브라우저가 HTML 렌더링]
| CSS 적용, JavaScript 실행
▼
[사용자에게 대시보드 표시]
```

1.2 DNS (Domain Name System) 란?

초등학생 설명

전화번호부입니다. "네이버"라고 말하면 "223.130.195.200"이라는 전화번호(IP)를 찾아주는 것.

동작 원리



AnchorIQ에서의 DNS

주소	해석	용도
localhost:3004	127.0.0.1:3004	Next.js 프론트엔드
localhost:8080	127.0.0.1:8080	Spring Boot 백엔드
localhost:7687	127.0.0.1:7687	Neo4j 데이터베이스
localhost:6380	127.0.0.1:6380	Redis 캐시
127.0.0.1:18789	로컬 OpenClaw AI	AI 게이트웨이

프로덕션에서는 `api.anchoriq.com` → AWS IP 주소처럼 실제 DNS가 동작합니다.

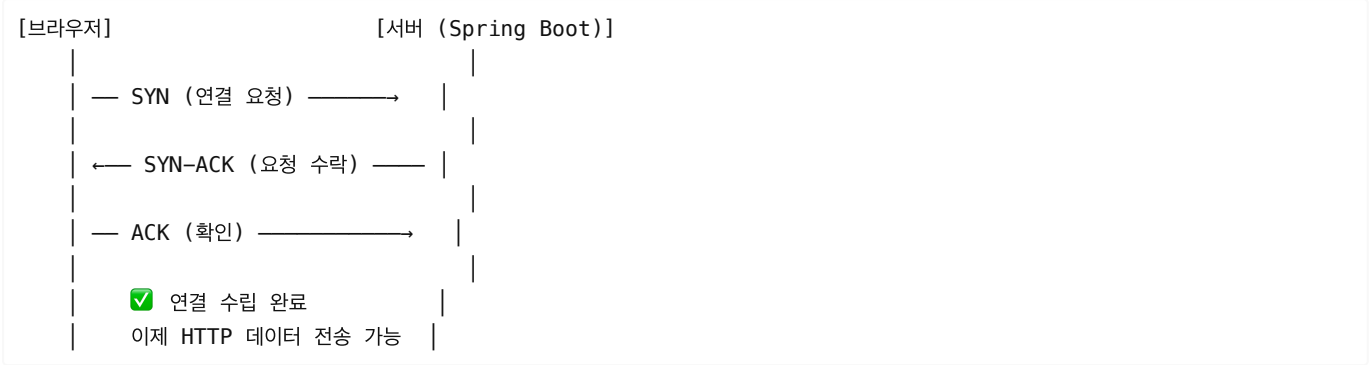
1.3 TCP/IP 란?

초등학생 설명

편지를 보내는 규칙입니다.

- **IP**: 주소 (어디로 보낼지) — 예: 127.0.0.1
- **TCP**: 배달 규칙 (순서대로, 빠짐없이, 확인 받고) — 포트 번호 포함

TCP 3-Way Handshake (연결 수립)



포트(Port)란?

하나의 컴퓨터(IP)에 여러 서비스가 돌아갈 수 있습니다. 포트는 **아파트 호수**와 같습니다.

127.0.0.1 (아파트 건물 = 내 컴퓨터)

- └─ :3004호 → Next.js 프론트엔드
- └─ :8080호 → Spring Boot 백엔드
- └─ :5433호 → PostgreSQL
- └─ :7474호 → Neo4j 브라우저
- └─ :7687호 → Neo4j Bolt 프로토콜
- └─ :6380호 → Redis
- └─ :29092호 → Kafka
- └─ :5678호 → n8n 자동화
- └─ :18789호 → OpenClaw AI

1.4 프로덕션 환경에서의 Nginx

Nginx란?

리버스 프록시(Reverse Proxy)입니다. 쉽게 말하면 **안내 데스크**. 모든 요청을 먼저 받아서 적절한 서버로 전달합니다.

왜 필요한가?

로컬에서는 포트 번호로 직접 접근하지만, 실제 서비스에서는:

사용자가 접속하는 주소:

- https://anchoriq.com (프론트엔드)
- https://anchoriq.com/api (백엔드 API)

실제 서버:

- 프론트엔드 → localhost:3004
- 백엔드 → localhost:8080

Nginx가 중간에서 라우팅:

Nginx 설정 예시 (프로덕션용)

```
server {  
    listen 443 ssl;                # HTTPS (암호화된 통신)  
    server_name anchoriq.com;      # 도메인  
  
    # SSL 인증서 (HTTPS를 위해 필요)
```

```

ssl_certificate /etc/ssl/cert.pem;
ssl_certificate_key /etc/ssl/key.pem;

# 프론트엔드 요청 → Next.js
location / {
    proxy_pass http://localhost:3004;
    proxy_set_header Host $host;
}

# API 요청 → Spring Boot
location /api/ {
    proxy_pass http://localhost:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;          # 실제 클라이언트 IP 전달
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

# WebSocket → Spring Boot
location /ws/ {
    proxy_pass http://localhost:8080;
    proxy_http_version 1.1;                          # WebSocket은 HTTP 1.1 필요
    proxy_set_header Upgrade $http_upgrade;          # 프로토콜 업그레이드
    proxy_set_header Connection "upgrade";
}
}

# HTTP → HTTPS 리다이렉트
server {
    listen 80;
    server_name anchoriq.com;
    return 301 https://$server_name$request_uri;    # 항상 HTTPS로
}

```

Nginx가 하는 일

역할	설명
리버스 프록시	외부 요청을 내부 서버로 전달
로드 밸런싱	서버가 여러 대일 때 요청 분산
SSL 종료	HTTPS 암호화/복호화 처리
정적 파일 서빙	CSS, JS, 이미지를 직접 전달 (서버 부담 감소)
보안	DDoS 방어, Rate Limiting, IP 차단

현재 AnchorIQ 로컬 vs 프로덕션

[로컬 개발] – Nginx 없음

브라우저 → localhost:3004 (프론트)

프론트 → localhost:8080/api (백엔드 직접)

[프로덕션] – Nginx 있음

브라우저 → anchoriq.com (Nginx)

Nginx → localhost:3004 (프론트) or localhost:8080 (API)

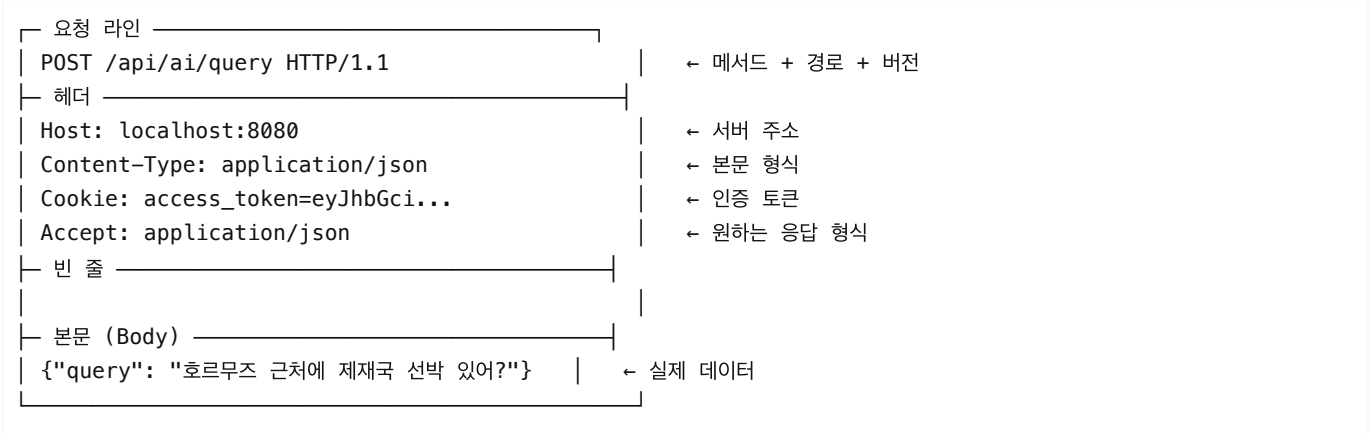
Part 2: HTTP 완전 정복

2.1 HTTP란?

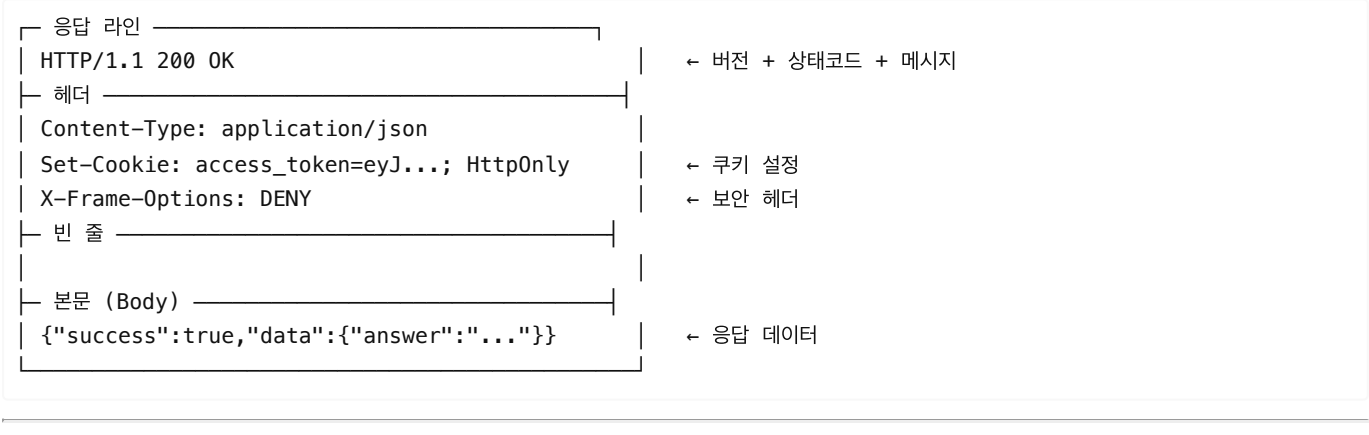
초등학생 설명

편지 양식입니다. 브라우저(클라이언트)가 서버에게 "이것 좀 줘" 또는 "이것 좀 저장해"라고 요청하는 형식.

HTTP 요청의 구조



HTTP 응답의 구조



2.2 HTTP 메서드 (GET, POST, PUT, DELETE)

비유: 도서관

메서드	도서관 비유	HTTP 설명	AnchorIQ 예시
GET	"이 책 좀 보여줘"	데이터 조회 (읽기만)	GET /api/ai/briefing — 일일 브리핑 조회
POST	"새 책 등록해줘"	데이터 생성 / 처리 요청	POST /api/ai/query — AI 질의 실행
PUT	"이 책 정보 수정해줘"	데이터 전체 수정	PUT /api/workflows/{id} — 워크플로우 수정
DELETE	"이 책 폐기해줘"	데이터 삭제	DELETE /api/bookmarks/{id} — 북마크 삭제

GET vs POST 차이 (중요!)

GET 요청:

GET /api/vessels?flag=KR&type=TANKER HTTP/1.1

- 데이터가 URL에 포함됨 (쿼리 파라미터)
- 브라우저 주소창에 보임
- 캐시 가능
- 본문(Body) 없음

POST 요청:

POST /api/ai/query HTTP/1.1

Content-Type: application/json

{"query": "호르무즈 근처에 제재국 선박 있어?"}

- 데이터가 본문(Body)에 포함됨
- URL에 안 보임
- 캐시 안 됨 (기본적으로)
- 민감한 데이터 전송에 적합

AnchorIQ의 실제 API 예시

인증:

- POST /api/auth/signup → 회원가입 (새 유저 생성)
- POST /api/auth/login → 로그인 (토큰 발급)
- POST /api/auth/refresh → 토큰 갱신
- POST /api/auth/logout → 로그아웃
- GET /api/auth/me → 내 정보 조회

AI:

- POST /api/ai/query → AI 질의 (자연어 → 답변)
- GET /api/ai/briefing → 일일 브리핑 조회
- GET /api/ai/usage → AI 사용량 조회
- POST /api/ai/whatif → What-if 시뮬레이션 실행
- GET /api/ai/whatif/templates → 시뮬레이션 템플릿 목록

선박:

- GET /api/vessels → 선박 목록
- GET /api/vessels/{imo} → 특정 선박 상세
- GET /api/vessels/{imo}/risk → 선박 리스크 분석

2.3 HTTP 상태 코드

서버가 "어떻게 처리했는지"를 숫자로 알려주는 것.

주요 상태 코드

코드	의미	비유	AnchorIQ에서
200 OK	성공	"여기 있어"	AI 질의 성공
201 Created	생성 성공	"등록했어"	회원가입 성공
400 Bad Request	잘못된 요청	"뭐라는 거야"	필수 필드 누락
401 Unauthorized	인증 실패	"너 누구야"	JWT 토큰 없음/만료
403 Forbidden	권한 없음	"넌 안 돼"	FREE 플랜이 PRO 기능 요청

404 Not Found	없음	"그런 거 없어"	존재하지 않는 선박 IMO
429 Too Many Requests	요청 과다	"좀 쉬어"	API 쿼터 초과
500 Internal Server Error	서버 오류	"내가 고장났어"	서버 버그

AnchorIQ의 통일된 응답 포맷

```
// 성공 (200)
{
  "success": true,
  "data": {
    "answer": "호르무즈 인근에 제재 대상국 관련 선박은...",
    "cypher": "MATCH (v:Vessel)...",
    "entities": []
  },
  "timestamp": "2026-04-04T04:43:17.126Z"
}

// 실패 (401)
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  },
  "timestamp": "2026-04-04T04:43:17.126Z"
}
```

2.4 HTTP vs HTTPS

	HTTP	HTTPS
포트	80	443
암호화	없음 (평문)	SSL/TLS 암호화
비유	엽서 (누구나 읽을 수 있음)	봉투 + 자물쇠 (열쇠 가진 사람만)
보안	패킷 스니핑에 취약	도청 불가능

HTTP: [브라우저] --"password=1234"--> [서버]
 ↑ 해커가 중간에서 볼 수 있음

HTTPS: [브라우저] --"#@\$%&*!@#"--> [서버]
 ↑ 암호화되어 있어 해커가 봐도 의미 없음

AnchorIQ 로컬 개발은 HTTP, 프로덕션 배포 시 HTTPS + Nginx SSL 적용.

Part 3: REST API란 무엇인가

3.1 API란?

초등학생 설명

식당 메뉴판입니다. 손님(프론트엔드)이 주방(백엔드)에 직접 들어갈 수 없으니, 메뉴판(API)을 보고 주문(요청)하면 음식(응답)이 나옵니다.

정식 정의

API(Application Programming Interface)란 소프트웨어 간 통신을 위한 약속된 인터페이스

```
[프론트엔드 (Next.js)]
|
|  "AI 질의 결과 줘"
|  POST /api/ai/query
|  {"query": "..."}
|
▼
[백엔드 API (Spring Boot)]
|
|  처리 후 응답
|  200 OK
|  {"success": true, "data": {...}}
|
▼
[프론트엔드가 화면에 표시]
```

3.2 REST란?

REpresentational **S**tate **T**ransfer — URL로 자원(Resource)을 표현하고, HTTP 메서드로 행동을 표현하는 아키텍처 스타일.

REST의 핵심 규칙

1. URL은 명사(자원)를 나타낸다

좋은 예:

```
GET /api/vessels          → 선박 목록 (자원: vessels)
GET /api/vessels/9863297 → 특정 선박 (자원: vessel with IMO 9863297)
POST /api/ai/query        → AI 질의 (자원: query)
```

나쁜 예:

```
GET /api/getVessels       → 동사 사용 (REST 위반)
POST /api/createUser      → 동사 사용 (REST 위반)
```

2. HTTP 메서드가 동사(행동)를 나타낸다

```
GET    /api/vessels      → 조회 (Read)
POST   /api/vessels      → 생성 (Create)
PUT    /api/vessels/123  → 수정 (Update)
DELETE /api/vessels/123  → 삭제 (Delete)
```

3. 상태를 저장하지 않는다 (Stateless)

각 요청은 독립적. 서버는 이전 요청을 기억하지 않음.
→ 그래서 매 요청마다 JWT 토큰을 같이 보내야 함.
→ "나 아까 로그인했잖아"가 안 됨. 매번 증명해야 함.

AnchorIQ REST API 전체 구조 (174개)

```
/api/auth/          → 인증 (signup, login, refresh, logout, me)
/api/vessels/       → 선박 (목록, 상세, 리스크, 관계, 이력)
```


/api/ports/	→ 항구 (혼잡도, ETA, 접안 이력)
/api/routes/	→ 항로 (초크포인트 통행, 사고)
/api/risks/	→ 리스크 (알림, 타임라인, 히트맵, 스코어)
/api/ai/	→ AI (질의, 브리핑, What-if, 리포트)
/api/graph/	→ 온톨로지 (그래프 탐색, 최단경로, 통계)
/api/sanctions/	→ 제재 (목록, 스크리닝, 업데이트)
/api/weather/	→ 날씨 (현재, 태풍, 예보)
/api/anomalies/	→ 이상 탐지 (AIS off, 항로 이탈)
/api/market/	→ 시장 (유가, 환율, 해운 지수)
/api/news/	→ 뉴스 (검색, 이벤트, 분쟁)
/api/workflows/	→ 자동화 (CRUD, 실행 이력)
/api/notifications/	→ 알림 (규칙, 설정, 이력)
/api/payments/	→ 결제 (구독, 취소, 이력)
/api/admin/	→ 관리자 (유저 관리, 시스템 상태)
/ws/	→ WebSocket (실시간 선박위치, 알림, 대시보드)

Part 4: Spring Boot & DDD 아키텍처

4.1 Spring Boot란?

초등학생 설명

레고 조립 세트입니다. 웹 서버, 데이터베이스 연결, 보안 등을 하나하나 만들지 않아도, 조립하면 바로 웹 애플리케이션이 완성됩니다.

Spring Boot가 해주는 것

기능	직접 하면	Spring Boot 쓰면
웹 서버	Tomcat 설치/설정/배포	자동 내장 (그냥 실행)
DB 연결	JDBC 드라이버, 커넥션 풀 설정	spring.datasource.url=... 한 줄
보안	필터, 세션, 암호화 직접 구현	Spring Security 라이브러리
직렬화	JSON 파싱 코드 작성	@RequestBody 어노테이션 하나

4.2 DDD (Domain-Driven Design) — AnchorIQ의 핵심 설계

초등학생 설명

프로그램을 현실 세계처럼 설계하는 방법입니다.

"선박"이 있으면 코드에도 `Vessel` 클래스가 있고, 현실에서 "선박이 리스크를 평가받는다"면 코드에서도 `vessel.evaluateRiskScore()` 를 호출합니다.

DDD 레이어 구조

AnchorIQ의 모든 모듈은 이 4개 레이어로 나뉩니다:

Controller (api/) - 문 앞 안내원	
"요청 받고 → 응답 보내는 것만"	
비즈니스 로직 없음. 입력 검증 + 응답 변환만.	
예: AiQueryController.java	
@PostMapping("/api/ai/query")	

```
public ResponseEntity<ApiResponse> query(...)
```

Application Service (application/) – 지휘관

"순서를 정하고 위임하는 것만"

비즈니스 판단 안 함. 오케스트레이션만.

예: AiQueryApplicationServiceImpl.java

1. 쿼터 확인
2. 캐시 확인
3. AI 서비스 호출
4. 사용량 증가
5. 로그 기록

Domain (domain/) – 전문가

"실제 판단과 규칙을 아는 곳"

비즈니스 로직이 여기에 있음!

예: Vessel.java

```
vessel.evaluateRiskScore(sanctionedCountries)
```

→ 제재국 연결: +40점

→ 고위험 깃발: +20점

→ 선령 20년 이상: +15점

Infrastructure (infrastructure/) – 실무자

"외부 시스템과 실제 통신"

DB 저장, API 호출, Kafka 전송 등

예: Neo4jVesselRepository.java

OpenClawClient.java

JwtAuthenticationFilter.java

왜 이렇게 나누는가?

나쁜 예 (모든 것을 Controller에):

```
@PostMapping("/api/ai/query")
public Response query(Request req) {
    // 쿼터 확인 로직 50줄
    // Redis 캐시 로직 30줄
    // AI 호출 로직 40줄
    // Neo4j 실행 로직 30줄
    // 응답 변환 20줄
    // 총 170줄짜리 거대한 메서드
}
```

좋은 예 (DDD 레이어 분리):

Controller → "요청 받아서 Service에 위임" (5줄)

Application Service → "순서 조율" (20줄)

Domain Service → "비즈니스 로직" (30줄)

Infrastructure → "외부 통신" (20줄)

→ 각각 독립적으로 테스트 가능

→ 변경 시 영향 범위가 작음

4.3 Entity란?

초등학생 설명

현실 세계의 물건을 코드로 표현한 것입니다. "선박"이라는 현실의 것을 `Vessel` 클래스로 만든 것.

AnchorIQ의 Vessel Entity (실제 코드)

```
// 파일: anchoriq-core/.../vessel/model/Vessel.java
// 패키지: com.anchoriq.core.domain.maritime.vessel.model

public class Vessel extends AggregateRoot {

    // === 속성 (현실 세계의 선박 정보) ===
    private Imo imo;           // IMO 번호 (선박 고유 식별자) - Value Object
    private Mmsi mmsi;        // MMSI 번호 - Value Object
    private String name;       // 선박명
    private Flag flag;         // 국적 (깃발) - Value Object
    private VesselType type;   // 선종 (TANKER, CONTAINER 등) - Enum
    private VesselStatus status; // 상태 (SAILING, MOORED 등) - Enum
    private Double deadweight; // 재화중량톤수
    private Integer buildYear; // 건조년도
    private int riskScore;     // 리스크 점수 (0~100)
    private Company company;   // 소유 회사

    // === 비즈니스 로직 (Entity가 직접 판단) ===

    // 리스크 점수 평가 - 핵심 비즈니스 로직!
    public int evaluateRiskScore(
        Set<String> sanctionedCountryCodes, // 제재국 목록
        Set<String> highRiskFlags) {       // 고위험 깃발 목록

        int score = 0;

        // 제재국 연결 회사 소유 → +40점
        if (isRegisteredInSanctionedCountry(sanctionedCountryCodes)) {
            score += 40;
        }

        // 편의치적(고위험 깃발) → +20점
        if (flag != null && highRiskFlags.contains(flag.value())) {
            score += 20;
        }

        // 선령 20년 이상 → +15점, 15년 이상 → +10점
        int age = calculateAge();
        if (age >= 20) score += 15;
        else if (age >= 15) score += 10;

        // 유조선 → +10점 (화물 위험도)
        if (isTanker()) score += 10;

        // AIS 이상 → +15점
        if (status == VesselStatus.NOT_UNDER_COMMAND) score += 15;
    }
}
```

```

        this.riskScore = Math.min(score, 100); // 최대 100점
        return this.riskScore;
    }

    // 상태 변경 - 유효한 전환만 허용
    public void changeStatus(VesselStatus newStatus) {
        status.validateTransitionTo(newStatus); // 잘못된 전환이면 예외 발생
        this.status = newStatus;
        // 이벤트 발행 → Kafka Consumer가 감지
    }
}

```

Entity vs Value Object vs Enum

구분	설명	예시	식별자
Entity	고유한 식별자가 있는 객체	Vessel, Company, Port	IMO, ID 등
Value Object	값으로 동등성 판단 (불변)	Imo, Mmsi, Flag, RiskScore	없음 (값 자체가 식별)
Enum	정해진 값 중 하나	VesselType, VesselStatus	없음

```

// Value Object 예시 - Imo
public record Imo(String value) {
    public Imo {
        // 생성 시 검증 (7자리 숫자인지)
        if (value == null || !value.matches("\\d{7}")) {
            throw new IllegalArgumentException("Invalid IMO: " + value);
        }
    }
}

// 왜 String 대신 Imo를 쓰는가?
// String imo = "not-a-valid-imo"; → 실수 가능, 컴파일러가 못 잡음
// Imo imo = new Imo("not-valid"); → 즉시 예외 발생! 실수 방지

```

4.4 Aggregate Root란?

초등학생 설명

대장입니다. 관련된 것들을 하나로 묶고, 외부에서는 반드시 대장을 통해서만 접근합니다.

Vessel (Aggregate Root = 대장)

- ├ PortCall (항구 방문 기록) - Vessel을 통해서만 접근
- ├ RouteHistory (항로 이력) - Vessel을 통해서만 접근
- └ RiskAssessment (리스크 평가) - Vessel을 통해서만 접근

외부에서:

```

vessel.addPortCall(port);           ← OK (대장을 통해 접근)
portCall.setVessel(vessel);         ← 금지! (대장 우회)

```

4.5 Repository 패턴

초등학생 설명

참고 관리자입니다. Entity를 저장하고 꺼내오는 역할.

```
[Application Service]
|
| "IMO 9863297 선박 가져와"
▼
[VesselRepository (인터페이스)] ← Domain 레이어 (추상)
|
▼
[Neo4jVesselRepository (구현체)] ← Infrastructure 레이어 (구체적)
|
▼
[Neo4j Database]
```

왜 인터페이스와 구현체를 분리하는가?

```
// Domain 레이어 - 인터페이스만 정의 (DB가 뭔지 모름)
public interface VesselRepository {
    Optional<Vessel> findByImo(Imo imo);
    void save(Vessel vessel);
}

// Infrastructure 레이어 - Neo4j 구현
@Repository
public interface Neo4jVesselRepository extends Neo4jRepository<Neo4jVesselNode, Long> {
    Optional<Neo4jVesselNode> findByImo(String imo);
}
```

장점: 나중에 Neo4j를 다른 DB로 바꿔도 Domain 코드는 안 건드릴. Infrastructure만 교체.

4.6 AnchorIQ 모듈 구조 (Gradle 멀티모듈)

```
anchoriq/backend/
├── anchoriq-core/           ← 순수 도메인 (Entity, VO, 인터페이스)
│   │                       외부 프레임워크 의존 없음!
│   └── domain/
│       ├── maritime/       ← vessel, port, route, company, country...
│       ├── intelligence/   ← risk, anomaly
│       ├── account/        ← user, subscription, payment
│       └── operation/      ← workflow, notification, bookmark
├── anchoriq-api/           ← 실행 모듈 (Controller, Security, DB 구현체)
│   ├── controller/         ← REST 엔드포인트
│   ├── application/        ← Application Service (오케스트레이션)
│   ├── dto/                ← Request/Response DTO
│   └── infrastructure/     ← DB 구현체, Security, 설정
├── anchoriq-ai/            ← AI 연동 모듈
│   ├── client/             ← OpenClawClient (LLM 호출)
│   ├── query/              ← NaturalLanguageQueryService
│   ├── briefing/           ← DailyBriefingService
│   ├── whatif/             ← WhatIfService
│   └── risk/                ← VesselRiskScorer 등
```

```
|— anchoriq-collector/      ← 데이터 수집 모듈
|   |— source/             ← 11개 API 수집기
|   |
|— anchoriq-automation/    ← 자동화 모듈
   |— n8n/                 ← n8n 워크플로우 연동
```

모듈 의존성 규칙 (빌드 레벨에서 강제)

```
anchoriq-core → 아무것도 의존하지 않음 (순수!)
anchoriq-api  → core, ai (사용 가능)
anchoriq-ai   → core (사용 가능), api (사용 불가!)
anchoriq-collector → core (사용 가능)
anchoriq-automation → core, ai (사용 가능)
```

core 모듈에 Spring 어노테이션(@Service, @Repository) 없음! 순수 Java만.

Part 5: 보안 — JWT 인증 흐름

5.1 JWT (JSON Web Token) 란?

초등학생 설명

놀이공원 손목 팔찌입니다. 입장할 때(로그인) 팔찌(JWT)를 받고, 놀이기구(API)를 탈 때마다 팔찌를 보여줍니다. 팔찌에는 "이름, 등급, 유효기간"이 적혀있습니다.

JWT의 구조

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3M4IiwiaWF0IjoiYXZ0ZXN0QGZyY2hvcmlxLmNvbSJ9.abc123
```

이것을 점(.)으로 나누면 3부분:

1. Header (헤더) — 암호화 알고리즘

```
{"alg": "HS256"}
```
2. Payload (내용물) — 사용자 정보

```
{
  "userId": 5,
  "email": "aitest@anchoriq.com",
  "role": "USER",
  "exp": 1712345678      ← 만료 시간
}
```
3. Signature (서명) — 위조 방지
HMAC-SHA256(header + payload, 비밀키)

왜 서버가 상태를 저장하지 않아도 되는가?

세션 방식 (옛날):

로그인 → 서버가 세션 ID 저장 → 매 요청마다 서버에서 세션 확인
문제: 서버가 많아지면 세션 공유 필요 (복잡)

JWT 방식 (AnchorIQ):

로그인 → 서버가 JWT 발급 → 이후 서버는 아무것도 저장하지 않음

매 요청마다: JWT의 서명만 검증 (비밀키로)
장점: 서버가 100대여도 각자 독립적으로 검증 가능

5.2 AnchorIQ 인증 전체 흐름

회원가입 → 로그인 → API 호출 → 토큰 갱신

[1단계: 회원가입]

```
POST /api/auth/signup
{"email": "aitest@anchoriq.com", "password": "Test1234!", "name": "AI Tester"}
|
▼
AuthController.signup()
|
▼
AuthApplicationService.signup()
| ① 이메일 중복 확인 (PostgreSQL)
| ② 비밀번호 BCrypt 해시 (원문 저장 안 함!)
|   "Test1234!" → "$2a$10$WKd.J2kf..."
| ③ User Entity 생성 + 저장
|
▼
201 Created
{"success": true, "data": {"id": 5, "email": "aitest@anchoriq.com"}}
```

[2단계: 로그인]

```
POST /api/auth/login
{"email": "aitest@anchoriq.com", "password": "Test1234!"}
|
▼
AuthController.login()
|
▼
AuthApplicationService.login()
| ① 이메일로 User 조회 (PostgreSQL)
| ② BCrypt로 비밀번호 검증
|   입력 "Test1234!" → 해시 → DB 해시와 비교
| ③ JWT Access Token 생성 (5시간 유효)
| ④ JWT Refresh Token 생성 (7일 유효)
| ⑤ HttpOnly Cookie에 토큰 설정
|
▼
200 OK
Set-Cookie: access_token=eyJhbG...; HttpOnly; Path=/; Max-Age=18000
Set-Cookie: refresh_token=eyJhbG...; HttpOnly; Path=/api/auth/refresh; Max-Age=604800
{"success": true, "data": {"tokenType": "Cookie", "expiresIn": 18000}}
```

[3단계: API 호출 (매 요청)]

```
POST /api/ai/query
Cookie: access_token=eyJhbG... ← 브라우저가 자동으로 보냄
{"query": "호르무즈 근처에 제재국 선박 있어?"}
|
```

```

▼
JwtAuthenticationFilter.doFilterInternal()
| ① Cookie에서 access_token 추출
| ② JwtTokenProvider.validateToken() - 서명 검증 + 만료 확인
| ③ 토큰에서 userId, email, role 추출
| ④ UserPrincipal 객체 생성
| ⑤ Spring SecurityContext에 인증 정보 설정
| ⑥ filterChain.doFilter() - 다음 단계로 진행
▼
AiQueryController.query()
| @AuthenticationPrincipal UserPrincipal ← 인증된 사용자 정보 주입
▼
... (AI 질의 처리)

```

[4단계: 토큰 만료 시 갱신]

```

Access Token 만료 (5시간 후) → 401 응답
|
▼
프론트엔드가 자동으로:
POST /api/auth/refresh
Cookie: refresh_token=eyJhbG...
|
▼
AuthController.refresh()
| ① Refresh Token 검증
| ② 새 Access Token 발급
| ③ 새 Refresh Token 발급 (Rotation)
▼
200 OK (새 토큰 쿠키 설정)

```

HttpOnly Cookie를 쓰는 이유

localStorage 방식 (위험):
 토큰이 JavaScript로 접근 가능
 → XSS 공격으로 토큰 탈취 가능!

```

document.cookie // 접근 불가 (HttpOnly)
localStorage.getItem('token') // 접근 가능! 위험!

```

HttpOnly Cookie 방식 (AnchorIQ):
 JavaScript로 접근 불가능
 브라우저가 자동으로 매 요청에 포함
 → XSS 공격으로 토큰 탈취 불가능!

Part 6: AnchorIQ AI 질의 전체 흐름 (코드 레벨)

6.1 전체 흐름도 (사용자 질문 → 답변)

```

사용자: "호르무즈 근처에 제재국 선박 있어?"
|
| POST /api/ai/query

```


| Cookie: access_token=eyJ...

| Body: {"query": "호르무즈 근처에 제재국 선박 있어?"}



[Layer 0] JwtAuthenticationFilter (보안)

파일: infrastructure/security/JwtAuthenticationFilter.java

- ① Cookie에서 access_token 추출
- ② 서명 검증 (HMAC-SHA256 + 비밀키)
- ③ 만료 시간 확인
- ④ userId=5, email=aitest@anchoriq.com, role=USER 추출
- ⑤ SecurityContext에 인증 정보 설정
- ⑥ 다음 필터로 전달



[Layer 1] AiQueryController (Controller 레이어)

파일: controller/ai/AiQueryController.java

```
@PostMapping("/api/ai/query")
public ResponseEntity<ApiResponse<AiQueryResponse>> query(
    @RequestBody AiQueryRequest request,          ← JSON → Java 객체 변환
    @AuthenticationPrincipal UserPrincipal user    ← 인증된 사용자 정보
) {
    Map<String, Object> result =
        aiQueryApplicationService.handleQuery(
            request.query(),    // "호르무즈 근처에 제재국 선박 있어?"
            user.userId()       // 5
        );
    return ResponseEntity.ok(ApiResponse.success(result));
}
```

역할: 입력 받기 + 응답 보내기 (판단 안 함)



[Layer 2] AiQueryApplicationServiceImpl (Application 레이어)

파일: application/ai/AiQueryApplicationServiceImpl.java

```
public Map<String, Object> handleQuery(String query, Long userId) {

    // ① API 쿼터 확인 (구독 플랜별 제한)
    checkApiQuota(userId);
    // FREE: 5회/일, PRO: 무제한
    // 초과 시 PlanLimitExceededException → 429 응답

    // ② Redis 캐시 확인 (5분 TTL)
```

```

String cacheKey = "ai:result:" + query.hashCode();
String cached = redisTemplate.opsForValue().get(cacheKey);
if (cached != null) {
    return parseCachedResult(cached); // 캐시 히트! AI 호출 안 함
}

// ㉓ AI 서비스 호출 (캐시 미스)
Map<String, Object> result = queryService.executeQuery(query);

// ㉔ 결과 캐싱 (5분)
redisTemplate.opsForValue().set(cacheKey, serialize(result), 5, TimeUnit.MINUTES);

// ㉕ API 사용량 증가
subscriptionService.incrementApiUsage(userId);

// ㉖ AI 결정 로그 기록 (Elasticsearch, 비동기)
aiDecisionLogConsumer.logDecision(query, result);

return result;
}

```

역할: 순서 조율 (판단 안 함, 위임만)



[Layer 3] NaturalLanguageQueryServiceImpl (Domain Service)

파일: anchoriq-ai/query/NaturalLanguageQueryServiceImpl.java

```

public Map<String, Object> executeQuery(String query) {

    // ㉓-A: 1차 AI 호출 - 자연어를 Cypher 쿼리로 변환
    String cypher = generateCypher(query);

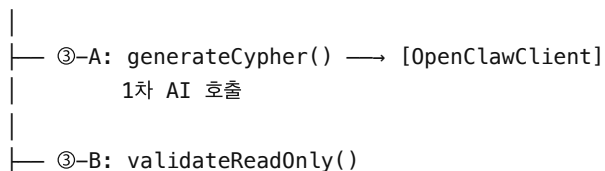
    // ㉓-B: Cypher 보안 검증 - 쓰기 쿼리 차단
    CypherQueryValidator.validateReadOnly(cypher);

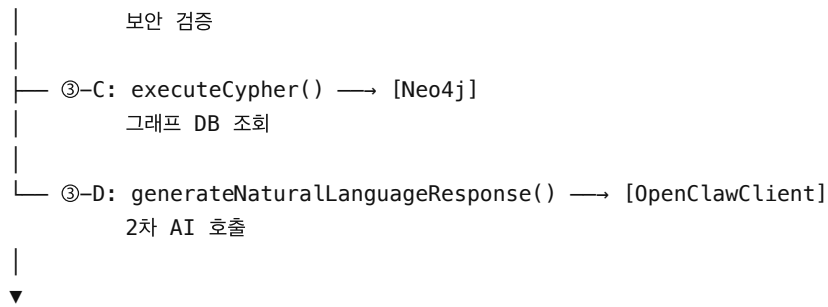
    // ㉓-C: Neo4j에서 Cypher 실행
    List<Map<String, Object>> results = executeCypher(cypher);

    // ㉓-D: 2차 AI 호출 - 결과를 자연어로 변환
    String answer = generateNaturalLanguageResponse(query, results);

    return Map.of(
        "cypher", cypher,
        "entities", results,
        "answer", answer
    );
}

```





[Layer 4-A] OpenClawClient – 1차 AI 호출 (Cypher 생성)

파일: anchoriq-ai/client/OpenClawClient.java

HTTP 요청:

POST http://127.0.0.1:18789/v1/chat/completions

Authorization: Bearer a0707484b483...

```
{
  "model": "openclaw:main",
  "messages": [
    {
      "role": "system",
      "content": "You are a Neo4j Cypher query expert...
                Nodes: Vessel(imo,mmsi,name,flag,type,status),
                Company(name,country),
                Country(code,name),
                Sanction(referenceNumber,targetName,type,active)...
                Relationships: OWNED_BY, REGISTERED_IN, SANCTIONED_BY...
                Rules: MATCH/RETURN only, NO writes"
    },
    {
      "role": "user",
      "content": "호르무즈 근처에 제재국 선박 있어?"
    }
  ],
  "temperature": 0.1,      ← 매우 낮음 = 정확한 Cypher 생성
  "max_tokens": 2000
}
```

AI 응답:

```
"MATCH (v:Vessel)-[:OWNED_BY]->(c:Company)
-[:REGISTERED_IN]->(co:Country)
-[:SANCTIONED_BY]->(s:Sanction),
(v)-[:SAILING_ON]->(r:Route)
-[:PASSES_THROUGH]->(cp:Chokepoint)
WHERE toLower(cp.name) CONTAINS 'hormuz'
RETURN v.name, v.imo, c.name, co.name, s.referenceNumber"
|
▼
```

[Layer 4-B] CypherQueryValidator – 보안 검증

차단 목록: CREATE, DELETE, DETACH, SET, REMOVE,
MERGE, DROP, CALL, LOAD CSV, FOREACH

허용: MATCH, RETURN, WHERE, WITH, ORDER BY, LIMIT

→ AI가 악의적 쿼리를 생성해도 실행 차단!



[Layer 4-C] Neo4j - 온톨로지 그래프 조회

Cypher 쿼리 실행:

```
(Vessel:알헤시라스)-[:OWNED_BY]->(Company:HMM)
-[:REGISTERED_IN]->(Country:한국)
-[:SANCTIONED_BY]-> ❌ 없음 (한국은 제재국이 아님)

(Vessel:SABITI)-[:OWNED_BY]->(Company:NITC)
-[:REGISTERED_IN]->(Country:이란)
-[:SANCTIONED_BY]->(Sanction:UN2231) ✅ 매칭!
BUT (SABITI)-[:SAILING_ON]->(Route)-[:PASSES_THROUGH]
->(Chokepoint: Hormuz?) → 현재 데이터에 없음
```

결과: 0건 (조건에 맞는 선박 없음)



[Layer 4-D] OpenClawClient - 2차 AI 호출 (응답 생성)

HTTP 요청:

```
POST http://127.0.0.1:18789/v1/chat/completions
{
  "messages": [
    {
      "role": "system",
      "content": "You are a maritime intelligence analyst.
                  Provide clear, concise natural language summary.
                  Focus on actionable insights. Under 200 words."
    },
    {
      "role": "user",
      "content": "Original query: 호르무즈 근처에 제재국 선박 있어?
                  Query results: [] (0 records)"
    }
  ],
  "temperature": 0.7      ← 약간 높음 = 자연스러운 한국어
}
```

AI 응답:

```
"현재 조회 결과상 호르무즈 인근에 제재 대상국 관련 선박은
확인되지 않았습니다. 실무적으로는..."
|
```



[Layer 5] 응답 반환 – 역순으로 올라감

NaturalLanguageQueryService → AiQueryApplicationService

→ Redis 캐싱 (5분)

→ API 사용량 증가 (PostgreSQL)

→ 로그 기록 (Elasticsearch)

→ AiQueryController → HTTP 200 OK 응답

최종 응답:

```
{
  "success": true,
  "data": {
    "answer": "현재 조회 결과상 호르무즈 인근에...",
    "entities": [],
    "cypher": "MATCH (v:Vessel)-[:OWNED_BY]->...",
    "remainingQueries": -1
  },
  "timestamp": "2026-04-04T04:43:17.126Z"
}
```



[프론트엔드가 화면에 표시]

Part 7: 데이터베이스 & 온톨로지

7.1 AnchorIQ의 4개 데이터베이스

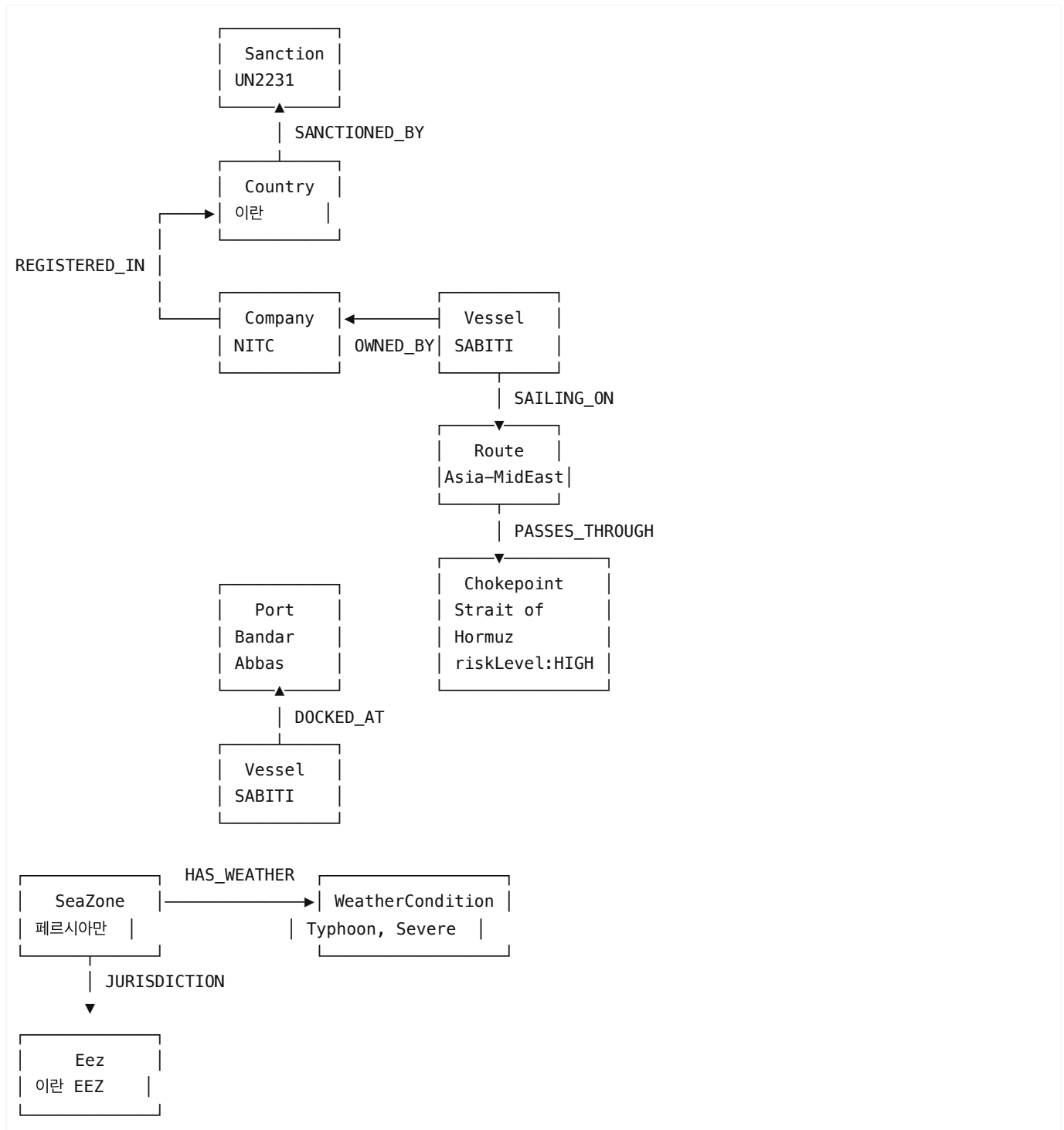
PostgreSQL (RDBMS)	Neo4j (Graph)	Redis (Cache)	Elasticsearch (Search)
포트: 5433	포트: 7687	포트: 6380	포트: 9200
테이블 기반 SQL	그래프 Cypher	Key-Value 명령어	문서 기반 JSON 쿼리
유저 결제 구독 감사 로그	선박 회사 국가 제재	AI 캐시 선박 위치 세션	뉴스 검색 AI 로그 감사 기록
강한 일관성 Tier 1	관계추론 Tier 2	초고속 Tier 3	전문검색 Tier 3

왜 4개나 쓰는가?

질문	최적 DB	이유
"이 유저의 결제 내역은?"	PostgreSQL	ACID 트랜잭션 필요 (돈!)

"이 선박의 소유 회사의 국가가 제재국인가?"	Neo4j	관계 탐색 (JOIN 6개 vs 1줄 Cypher)
"이 질의 5분 전에도 했는데?"	Redis	밀리초 응답 (캐시)
"최근 뉴스에서 '호르무즈'가 나온 기사는?"	Elasticsearch	전문 검색

7.2 Neo4j 온톨로지 스키마 (AnchorIQ 실제 구조)



4-hop 관계 탐색 예시

"이 선박이 제재국과 연결되어 있는가?"를 4번의 관계 이동으로 답변.
SQL이면 4개 테이블 JOIN. Cypher면 한 줄.

Tier	대상	전략	실패 시
Tier 1	PostgreSQL (돈, 유저)	@Transactional — 강한 일관성	즉시 롤백
Tier 2	Neo4j (온톨로지)	Kafka 최종 일관성	이벤트 재처리
Tier 3	Redis, ES (캐시, 로그)	실패 무시	로그만 남김

```
redisTemplate.opsForValue().set(cacheKey, result, 5, MINUTES);
aiDecisionLogConsumer.logDecision(...);
→ 실패해도 서비스에 영향 없음 (다음에 다시 캐싱하면 됨)
```

장점: 각각 독립적. 하나가 죽어도 나머지는 동작. 나중에 재처리 가능.

AnchorIQ Kafka 토픽 8개

토픽	Producer	Consumer	용도
ais-positions	AIS WebSocket	Redis, Neo4j, AI	선박 실시간 위치
weather-events	Open-Meteo API	Neo4j, 알림	기상 정보
sanction-updates	UN API	Neo4j, AI	제재 목록 변경
risk-alerts	AI 엔진	n8n, 알림	리스크 알림
news-events	뉴스 수집기	ES, AI	뉴스 데이터
market-data	유가/환율 API	Redis, Neo4j	시장 데이터
geopolitical-events	지정학 수집기	Neo4j, AI	분쟁/긴장
port-updates	항구 API	Neo4j, Redis	항구 혼잡도

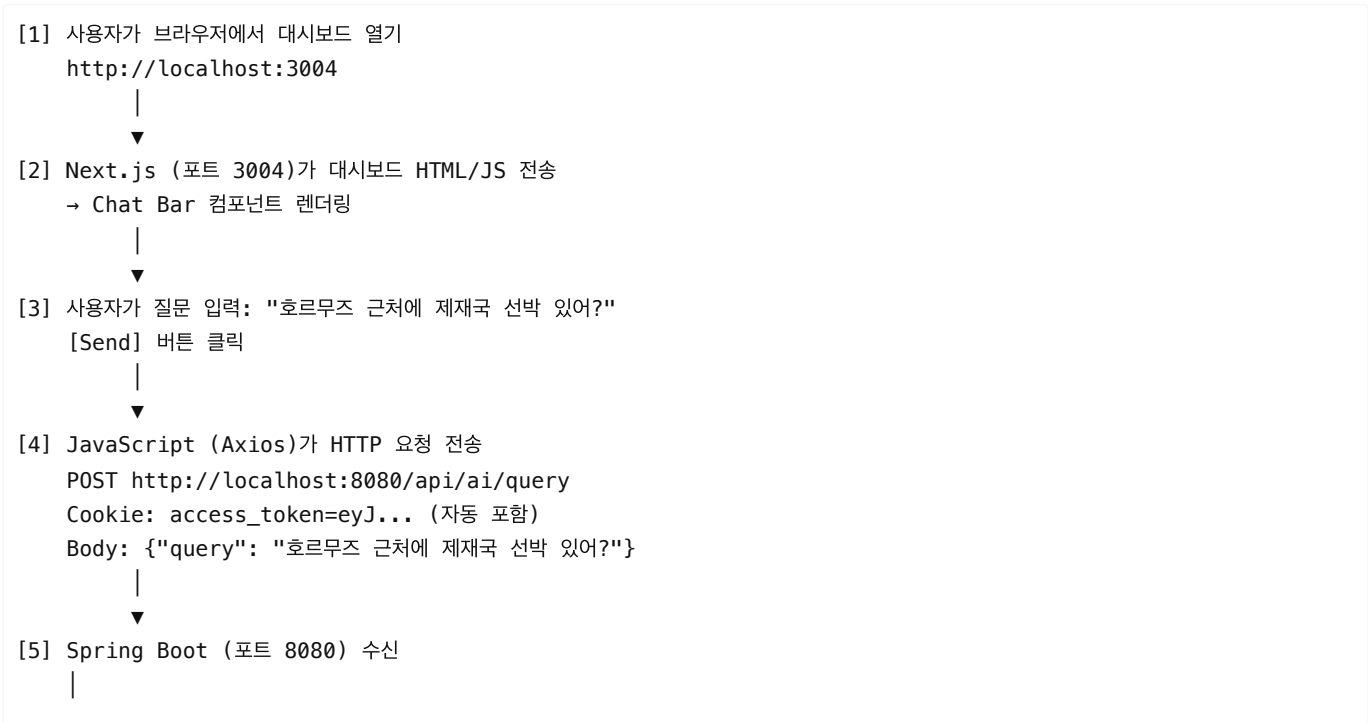
에러 처리: Dead Letter Topic (DLT)

메시지 처리 실패 시:
1차 시도 실패 → 1초 대기 → 재시도
2차 시도 실패 → 1초 대기 → 재시도
3차 시도 실패 → Dead Letter Topic으로 이동

DLT에 쌓인 메시지:
→ 원인 분석 후 수동 재처리
→ 전체 파이프라인은 멈추지 않음!

Part 9: 프론트엔드에서 백엔드까지 전체 연결

9.1 사용자가 AI 질문하는 전체 여정



- └ [Security] JWT 검증 ✅
- └ [Controller] 요청 라우팅
- └ [Application] 쿼터 확인 + 캐시 확인
- └ [AI Service] 1차 AI → Cypher 생성
- └ [Validator] Cypher 보안 검증
- └ [Neo4j] 온톨로지 그래프 조회
- └ [AI Service] 2차 AI → 자연어 응답 생성
- └ [Redis] 결과 캐싱 (5분)
- └ [PostgreSQL] API 사용량 증가
- └ [Elasticsearch] 로그 기록 (비동기)

|



[6] HTTP 200 응답 반환

```
{
  "success": true,
  "data": {
    "answer": "현재 조회 결과상 호르무즈 인근에...",
    "cypher": "MATCH (v:Vessel)...",
    "entities": [...]
  }
}
```

|



[7] JavaScript가 응답을 파싱하여 화면에 표시

- └ answer → Chat Bar 아래에 텍스트 표시
- └ entities → "지도에서 보기" 버튼 생성
- └ cypher → 개발자 도구에서 확인 가능

|



[8] 사용자가 답변을 확인

"현재 조회 결과상 호르무즈 인근에 제재 대상국 관련 선박은
확인되지 않았습니다..."

Part 10: 운영 인프라 — Docker, 모니터링

10.1 Docker란?

초등학생 설명

도시락 통입니다. 프로그램 + 필요한 모든 것(라이브러리, 설정)을 하나의 통(컨테이너)에 담아서, 어디서든 똑같이 실행됩니다.

왜 필요한가?

Docker 없이:

"내 컴퓨터에서는 되는데..."
→ Java 버전 다름, Neo4j 설정 다름, OS 다름...

Docker 있어:

"docker compose up" 한 번이면 모든 환경 동일
→ 개발, 테스트, 운영 환경 모두 같은 컨테이너

AnchorIQ Docker Compose (11개 서비스)

```

services:
  postgresql:      # 유저/결제 DB
    image: postgres:16
    ports: ["5433:5432"]

  neo4j:           # 온톨로지 그래프 DB
    image: neo4j:5-community
    ports: ["7474:7474", "7687:7687"]

  redis:           # 실시간 캐시
    image: redis:7-alpine
    ports: ["6380:6379"]

  kafka:           # 이벤트 스트리밍
    image: confluentinc/cp-kafka:7.6.0
    ports: ["29092:29092"]

  elasticsearch:  # 전문 검색
    image: elasticsearch:8.13.0
    ports: ["9200:9200"]

  n8n:             # 자동화 워크플로우
    image: n8nio/n8n
    ports: ["5678:5678"]

  prometheus:     # 메트릭 수집
    image: prom/prometheus
    ports: ["9090:9090"]

  grafana:        # 모니터링 대시보드
    image: grafana/grafana
    ports: ["3001:3000"]

```

프로필별 실행

```

# 최소 (개발용)
docker compose --profile core up -d
→ PostgreSQL + Redis만

# 데이터 처리 포함
docker compose --profile core --profile data up -d
→ + Kafka + Elasticsearch

# 전체 (운영용)
docker compose --profile full up -d
→ 11개 전부

```

10.2 모니터링 (Prometheus + Grafana)

```

[Spring Boot]
| /actuator/prometheus
| (메트릭 노출: CPU, 메모리, 요청 수, 응답 시간...)
▼

```

- [Prometheus] (메트릭 수집기, 15초 간격)

| 수집한 데이터 저장

▼

[Grafana] (시각화 대시보드)

| 그래프, 알람 설정

▼

[관리자가 모니터링]

- API 응답 시간 그래프

- 에러율 추이

- JVM 메모리 사용량

- Kafka Consumer Lag

- Neo4j 쿼리 성능

부록: 용어 총정리

용어	한줄 설명
DNS	도메인(naver.com)을 IP(223.130.195.200)로 변환하는 시스템
TCP	데이터를 순서대로, 빠짐없이, 확인 받으며 전송하는 프로토콜
HTTP	브라우저와 서버 간 통신 규약 (요청-응답 구조)
HTTPS	HTTP + SSL/TLS 암호화 (보안 통신)
REST	URL로 자원, HTTP 메서드로 행동을 표현하는 API 설계 방식
GET	데이터 조회 요청 (읽기만, 변경 없음)
POST	데이터 생성/처리 요청 (서버 상태 변경)
PUT	데이터 전체 수정 요청
DELETE	데이터 삭제 요청
상태 코드	서버 응답 결과 (200=성공, 401=인증실패, 500=서버오류)
JSON	데이터 교환 형식 {"key": "value"}
Nginx	리버스 프록시/로드밸런서 (안내 데스크)
SSL/TLS	통신 암호화 프로토콜 (HTTPS의 S)
Spring Boot	Java 웹 애플리케이션 프레임워크 (레고 조립 세트)
DDD	현실 세계를 그대로 코드로 설계하는 방법론
Entity	고유 식별자가 있는 도메인 객체 (Vessel, User)
Value Object	값으로 동등성 판단하는 불변 객체 (Imo, Email)
Aggregate Root	관련 Entity 묶음의 대장 (외부 접근 진입점)
Repository	Entity 저장/조회 인터페이스 (창고 관리자)
Domain Service	여러 Entity에 걸친 비즈니스 로직
Application Service	오케스트레이션만 (비즈니스 판단 안 함)
Controller	HTTP 요청 수신 + 응답 반환 (문 앞 안내원)

DTO	레이어 간 데이터 전달 객체 (Request/Response)
JWT	JSON Web Token (인증 토큰, 놀이공원 팔찌)
HttpOnly Cookie	JavaScript 접근 불가 쿠키 (XSS 방어)
BCrypt	비밀번호 해시 알고리즘 (원문 복원 불가)
CORS	다른 도메인 간 요청 허용 정책
Neo4j	그래프 데이터베이스 (노드 + 관계)
Cypher	Neo4j의 쿼리 언어 (SQL의 그래프 버전)
온톨로지	도메인 개념과 관계의 형식적 정의 (설계도)
지식그래프	온톨로지에 실제 데이터를 넣은 것 (건물)
Redis	인메모리 Key-Value 스토어 (초고속 캐시)
Elasticsearch	전문 검색 엔진 (뉴스, 로그 검색)
Kafka	분산 이벤트 스트리밍 플랫폼 (우체국)
Producer	Kafka에 메시지를 보내는 쪽
Consumer	Kafka에서 메시지를 가져가는 쪽
DLT	Dead Letter Topic (실패 메시지 격리 저장소)
Docker	컨테이너 기반 가상화 (도시락 통)
Docker Compose	여러 컨테이너를 한 번에 실행하는 도구
Prometheus	메트릭 수집/저장 시스템
Grafana	모니터링 시각화 대시보드
LLM	Large Language Model (대규모 언어 모델, GPT 등)
RAG	검색 증강 생성 (검색 후 AI가 답변)
Hallucination	AI가 사실이 아닌 것을 생성하는 현상
OpenClaw	OpenAI 호환 로컬 AI 게이트웨이

이 문서의 모든 내용은 AnchorIQ의 실제 코드와 설정을 기반으로 작성되었습니다. 면접에서 "이 시스템이 어떻게 동작하나요?"라는 질문에 Part 6의 전체 흐름을 설명하면 됩니다.